

01_Data_Preparation: MF4 $\rightarrow$ Cleaned 10 ms Dataset

Validation_Aero_Test

February 12, 2026

1 Overview

This document describes the current data-preparation pipeline implemented in `Code/01_Data_Preparation/`.

Goal Create a single analysis-ready CSV with a fixed sample raster (default: 10 ms) and stable column names for downstream steps (run extraction and drag calculation).

Primary outputs

- Per-channel exports: `Outputs/01_Data_Preparation/Signals/*.csv`
- Canonical cleaned dataset: `Outputs/01_Data_Preparation/Aero_Validation.Signals_cleaned.10ms.c`

Entry points

- `export_channels_10ms_to_signals.py`: MF4 $\rightarrow$ per-channel CSVs.
- `build_cleaned_signals_10ms.py`: per-channel CSVs $\rightarrow$ cleaned, combined dataset with derived signals.

2 Inputs

- Raw logging data: `Data/<session>/Stint*.mf4`
- Channel list: `Data/channels_extraction.txt` (lines, # for comments)

3 Step 1: Export per-channel CSVs (fixed raster)

Implemented in `Code/01_Data_Preparation/export_channels_10ms_to_signals.py`.

Concatenation strategy

Each `Stint*.mf4` is treated as a separate segment:

- Timestamps are shifted to start at 0 within the stint.
- Signals are interpolated onto a uniform grid with spacing Δt (default: 0.01 s).
- Stints are concatenated on a continuous time axis with an optional single- Δt gap between stints (enabled by default).

Per-channel output schema

For each requested channel c , the exporter writes:

- File: `Outputs/01_Data_Preparation/Signals/<channel>.csv`
- Columns: `time` and `<channel_name>` (numeric, NaN outside signal support)

4 Step 2: Combine to the canonical cleaned dataset

Implemented in `Code/01_Data_Preparation/build_cleaned_signals_10ms.py`.

Join semantics

All per-channel CSVs are read and joined on the time column using an outer join. The resulting table is sorted by time and uses:

- index name `t_s` (seconds)
- output column `t_s` after resetting the index

Canonical column naming

The script renames a small set of known extracted channels to stable analysis names with units, e.g. `SBG_GPS1_VELOCITY_N` $\rightarrow$ `GPS1_VEL_N_ms`. This renaming is intentionally explicit to avoid accidental mismatches when channel naming changes.

5 Derived signals and notation

We use the following time-synchronized signals (after column sanitization):

- t [s]: timestamp `t_s`.
- $\mathbf{a}_{\text{IMU}} = (a_x, a_y, a_z)$ [m/s^2]: IMU accelerations `IMU_ACCEL_X_ms2`, `IMU_ACCEL_Y_ms2`, `IMU_ACCEL_Z_ms2`.
- v_{gps} [m/s]: GPS speed magnitude `GPS1_VEL_MAG_ms`.
- v_{wheel} [m/s]: wheelspeed magnitude, computed from `WHEEL_SPEED_kmh` by $v_{\text{wheel}} = \text{km/h} \cdot \frac{1000}{3600}$.
- θ [deg]: steering angle `ICU_SteeringAngle_deg`.

The derived signals are produced by `aero_validation.signals:add_derived_signals`.

6 Zero-Velocity Bias Removal

Bias and gravity components are estimated from near-stationary samples and subtracted per axis:

1. Build a zero-velocity mask using available speed signals:
 $\mathcal{Z} = \{i \mid v_{\text{gps}}[i] < 0.05 \text{ m/s}\}$ if GPS exists; if wheelspeed exists, combine as $\mathcal{Z} = \{i \mid v_{\text{gps}}[i] < 0.05 \wedge v_{\text{wheel}}[i] < 0.05\}$.
2. Estimate per-axis stationary bias as medians over $\mathcal{Z}$ (fallback to global median if $|\mathcal{Z}|$ is small):
 $b_x = \text{median}\{a_x[i] : i \in \mathcal{Z}\}$, similarly b_y, b_z .
3. Form bias-corrected IMU accelerations: $a'_x = a_x - b_x$, $a'_y = a_y - b_y$, $a'_z = a_z - b_z$.

7 Smoothing and dv/dt

To obtain a stable numerical derivative of speed, we apply light smoothing and then differentiate:

1. Compute median sample period Δt_{med} from t and set a smoothing window of approximately 0.5 s (e.g., a centered rolling mean of $\approx \frac{0.5}{\Delta t_{\text{med}}}$ samples).
2. Smooth the speed trace $v(t)$ (prefer v_{gps} , fallback to v_{wheel}) to obtain $\tilde{v}(t)$.
3. Compute the numerical derivative using a gradient operator:

$$\frac{dv}{dt}(t) \approx \text{gradient}(\tilde{v}(t), t).$$

8 Straight and Moving Segment Masking

We restrict learning to segments where the vehicle is moving and approximately straight:

- Moving mask: $v(t) > 0.5 \text{ m/s}$.
- Straight mask: $|\theta(t)| \leq 5^\circ$ when steering is available; otherwise skipped.

The final validity mask is the conjunction of finiteness checks and the above conditions.

9 Data-Driven Longitudinal Axis Estimation

On valid samples, we estimate the longitudinal direction in the IMU frame by regressing the bias-corrected IMU signals onto dv/dt :

$$\begin{aligned} \text{Given } A &= \begin{bmatrix} a'_x & a'_y & a'_z \end{bmatrix} \in \mathbb{R}^{N \times 3}, \\ y &= \left(\frac{dv}{dt} \right) \in \mathbb{R}^N, \\ \text{find } \mathbf{w} \in \mathbb{R}^3 &\text{ such that } A \mathbf{w} \approx y \text{ (least squares).} \end{aligned}$$

We solve

$$\min_{\mathbf{w}} \|A\mathbf{w} - y\|_2^2,$$

and obtain $\mathbf{w}$ via a standard linear least squares solver. If insufficient valid data is available, we fall back to $\mathbf{w} = (1, 0, 0)$.

Longitudinal Acceleration Projection The longitudinal acceleration signal is then computed as

$$a_{\text{long}}(t) = w_x a'_x(t) + w_y a'_y(t) + w_z a'_z(t).$$

10 Notes

- **Resampling:** interpolation is performed independently per channel; NaNs indicate missing data or out-of-range timestamps.
- **Bias removal:** the “stationary” mask uses a strict low-speed threshold ($< 0.05 \text{ m/s}$) and falls back to global medians if too few stationary samples exist.
- **Longitudinal axis fit:** the least-squares weights are only estimated on straight + moving segments to avoid lateral acceleration contamination.

11 Example outputs

The figures below are generated directly from the canonical cleaned dataset `Outputs/01_Data_Preparation/Aer`

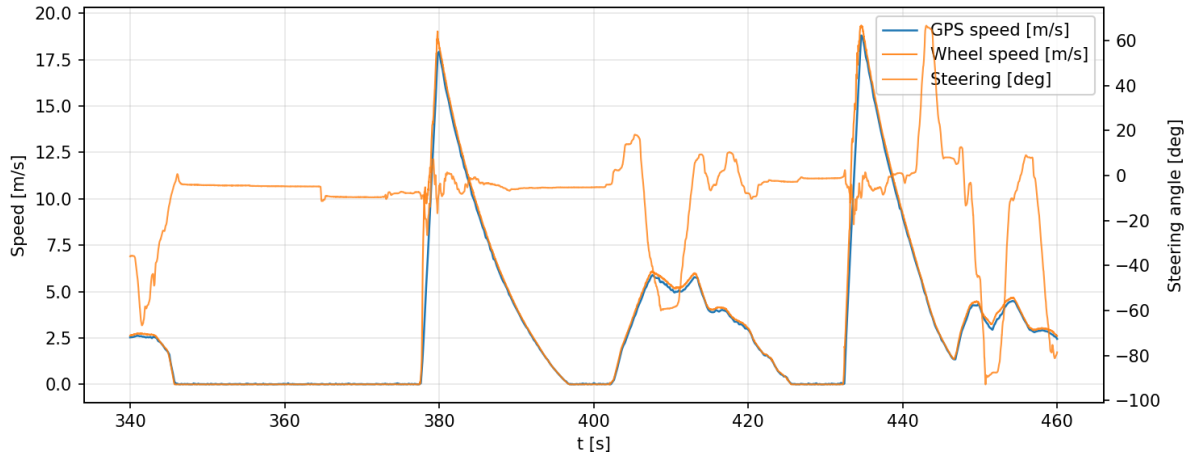


Figure 1: Example time window from the cleaned dataset showing GPS speed, wheel speed (converted), and steering angle.

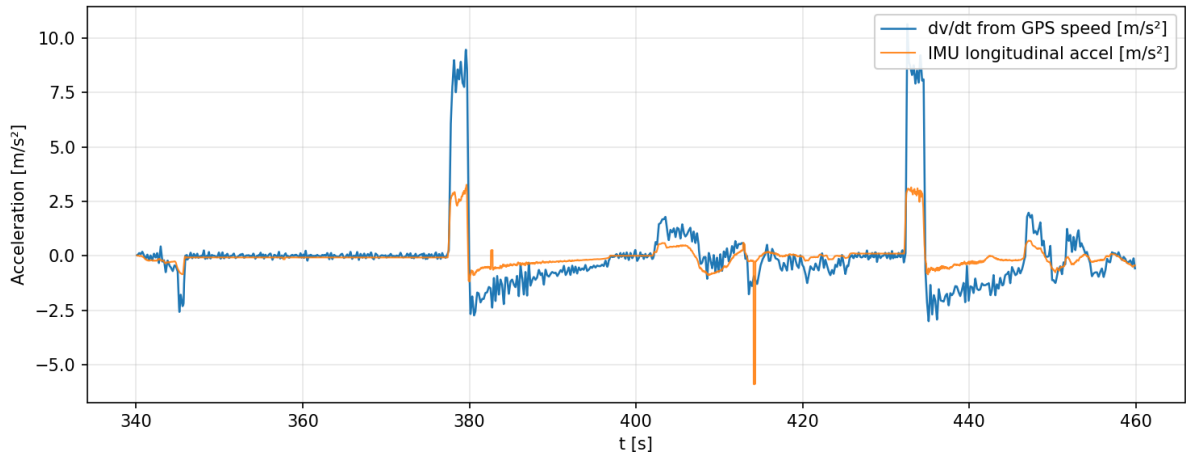


Figure 2: Example comparison between dv/dt (computed from GPS speed) and the derived longitudinal IMU acceleration signal `IMU_ACCEL_LONG_ms2`.

12 Implementation references

- Export: `Code/01_Data_Preparation/export_channels_10ms_to_signals.py`
- Combine + derived signals: `Code/01_Data_Preparation/build_cleaned_signals_10ms.py`
- Derived signal implementation: `aero_validation/signals.py` (function `add_derived_signals`)